

FinPlanner - Bugs Found & Fixes Applied

Date: January 1, 2026

Testing Phase: End-to-End Verification

Bug #1: SelectItem Component Structure Error



Issue Details

- **Severity:** HIGH (Blocking)
- **Status:** ☒ FIXED (Code updated, requires rebuild)
- **Impact:** Cannot open Add Bill dialog and other dialogs with dropdowns

Error Message

Error: Element type **is** invalid: expected a string (**for** built-**in** components) or a **class/function** (**for** composite components) but got: object.

Root Cause

SelectItem components contained nested `<div>` elements, which violates React component structure rules. The shadcn/ui SelectItem component expects plain text or simple content, not complex JSX structures.

Files Fixed (8 files)

1. components/add-bill-dialog.tsx

Before:

```
<SelectItem key={category.id} value={category.id}>
  <div className="flex items-center">
    <div className="w-3 h-3 rounded-full mr-2" style={{ backgroundColor: category.color }} />
    {category.name}
  </div>
</SelectItem>
```

After:

```
<SelectItem key={category.id} value={category.id}>
  {category.name}
</SelectItem>
```

2. components/add-transaction-dialog.tsx

Before:

```

<SelectedItem value="INCOME">
  <div className="flex items-center space-x-2">
    <div className="w-3 h-3 rounded-full bg-emerald-500"></div>
    <span>Income</span>
  </div>
</SelectedItem>

```

After:

```

<SelectedItem value="INCOME">Income</SelectedItem>
<SelectedItem value="EXPENSE">Expense</SelectedItem>
<SelectedItem value="TRANSFER">Transfer</SelectedItem>

```

Also fixed category SelectItems:

```

<SelectedItem key={category.id} value={category.id}>
  {category.name}
</SelectedItem>

```

3. components/global-command-bar.tsx

Before:

```

<SelectedItem key={category.id} value={category.id}>
  <div className="flex items-center gap-2">
    <div className="w-3 h-3 rounded-full" style={{ backgroundColor:
category.color }} />
    {category.name}
  </div>
</SelectedItem>

```

After:

```

<SelectedItem key={category.id} value={category.id}>
  {category.name}
</SelectedItem>

```

4. components/transaction-filters.tsx

Before:

```

<SelectedItem key={category.id} value={category.id}>
  <div className="flex items-center space-x-2">
    <div className="w-3 h-3 rounded-full" style={{ backgroundColor:
category.color }} />
    <span>{category.name}</span>
  </div>
</SelectedItem>

```

After:

```
<SelectedItem key={category.id} value={category.id}>
  {category.name}
</SelectedItem>
```

5. components/sms-parser-dialog.tsx

Before:

```
<SelectedItem key={category.id} value={category.id}>
  <div className="flex items-center space-x-2">
    <div className="w-3 h-3 rounded-full" style={{ backgroundColor:
category.color }} />
    <span>{category.name}</span>
    {category.name === parsedTransaction.suggestedCategory && (
      <Badge variant="secondary" className="ml-2 text-xs">Suggested</Badge>
    )}
  </div>
</SelectedItem>
```

After:

```
<SelectedItem key={category.id} value={category.id}>
  {category.name}
  {category.name === parsedTransaction.suggestedCategory && ' (Suggested)'}
</SelectedItem>
```

6. components/edit-goal-dialog.tsx

Before:

```
<SelectedItem key={category.id} value={category.id}>
  <div className="flex items-center gap-2">
    <div className="w-3 h-3 rounded-full" style={{ backgroundColor:
category.color }} />
    {category.name}
  </div>
</SelectedItem>
```

After:

```
<SelectedItem key={category.id} value={category.id}>
  {category.name}
</SelectedItem>
```

7. components/goal-selection-dialog.tsx

Before:

```

<SelectedItem key={goal.id} value={goal.id}>
  <div className="flex items-center justify-between w-full">
    <div className="flex items-center gap-2">
      <Target className="h-4 w-4" />
      <span>{goal.name}</span>
    </div>
    <div className="text-right text-sm">
      <div className="font-medium">{formatCurrency(goal.targetAmount)}</div>
      <Badge className={goalTypeColors[goal.goalType]} variant="secondary">
        {goal.goalType.replace('_', ' ')}
      </Badge>
    </div>
  </div>
</SelectedItem>

```

After:

```

<SelectedItem key={goal.id} value={goal.id}>
  {goal.name} - {formatCurrency(goal.targetAmount)}
</SelectedItem>

```

8. components/link-investment-dialog.tsx

Before:

```

<SelectedItem key={investment.id} value={investment.id}>
  <div className="flex items-center justify-between w-full">
    <div className="flex items-center gap-2">
      <TrendingUp className="h-4 w-4" />
      <span>{investment.name}</span>
    </div>
    <div className="text-right text-sm">
      <div className="font-medium">{formatCurrency(investment.currentValue)}</div>
      <Badge className={assetClassColors[investment.assetClass]} variant="secondary">
        {investment.assetClass.replace('_', ' ')}
      </Badge>
    </div>
  </div>
</SelectedItem>

```

After:

```

<SelectedItem key={investment.id} value={investment.id}>
  {investment.name} - {formatCurrency(investment.currentValue)}
</SelectedItem>

```

Next Steps

1. Clear Next.js build cache: `rm -rf .next`
 2. Restart development server: `npm run dev`
 3. Test all dialogs to verify fixes
-

Bug #2: Missing AI Module (abacusai)



Issue Details

- **Severity:** CRITICAL (Blocking)
- **Status:** ⌚ PENDING (Requires implementation)
- **Impact:** SMS parsing and command bar parsing non-functional

Error Message

```
Module not found: Can't resolve 'abacusai'  
Error parsing command: Error: Cannot find module 'abacusai'
```

Root Cause

The application code references the `abacusai` npm package which:

1. Is not installed in the project
2. Does not exist in the npm registry
3. Was likely intended to be a custom SDK or different AI service

Affected Files

- `app/api/ai/parse-sms/route.ts`
- `app/api/ai/parse-command/route.ts`

Current Code (Non-functional)

```
// Use Abacus.AI SDK for LLM parsing  
const AbacusAI = require('abacusai')  
const client = new AbacusAI.ApiClient()
```

Recommended Solutions

Option A: OpenAI GPT-4 Integration (Recommended)

Install:

```
npm install openai
```

Implementation:

```
import OpenAI from 'openai'

const openai = new OpenAI({
  apiKey: process.env.OPENAI_API_KEY
})

const completion = await openai.chat.completions.create({
  model: "gpt-4",
  messages: [
    { role: "system", content: systemPrompt },
    { role: "user", content: input }
  ],
  response_format: { type: "json_object" }
})

const parsed = JSON.parse(completion.choices[0].message.content)
```

Environment Variable:

```
OPENAI_API_KEY=sk-...
```

Option B: Anthropic Claude Integration

Install:

```
npm install @anthropic-ai/sdk
```

Implementation:

```
import Anthropic from '@anthropic-ai/sdk'

const anthropic = new Anthropic({
  apiKey: process.env.ANTHROPIC_API_KEY
})

const message = await anthropic.messages.create({
  model: "claude-3-5-sonnet-20241022",
  max_tokens: 1024,
  messages: [
    { role: "user", content: `${systemPrompt}\n\n${input}` }
  ]
})

const parsed = JSON.parse(message.content[0].text)
```

Environment Variable:

```
ANTHROPIC_API_KEY=sk-ant-...
```

Option C: Rule-Based Parsing (Fallback)

For SMS Parsing:

```

function parseSmsFallback(smsText: string) {
  const patterns = {
    debit: /(?:debited|spent|paid)\s+(?:rs\.|₹)\s*(\d+(?:,\d+)*(?:\.\d+)?)/i,
    credit: /(?:credited|received)\s+(?:rs\.|₹)\s*(\d+(?:,\d+)*(?:\.\d+)?)/i,
    date: /(\d{2})-(\w{3})-(\d{2})/,
    merchant: /(?:at|from|to)\s+([A-Z][A-Za-z\s]+)/
  }

  // Extract amount
  const debitMatch = smsText.match(patterns.debit)
  const creditMatch = smsText.match(patterns.credit)
  const amount = debitMatch?.[1] || creditMatch?.[1]
  const type = debitMatch ? 'EXPENSE' : 'INCOME'

  // Extract date
  const dateMatch = smsText.match(patterns.date)
  const date = dateMatch ? parseDate(dateMatch[0]) : new Date()

  // Extract merchant
  const merchantMatch = smsText.match(patterns.merchant)
  const merchant = merchantMatch?.[1]?.trim()

  // Suggest category based on keywords
  const category = suggestCategory(smsText)

  return {
    amount: parseFloat(amount?.replace(/,/g, '') || '0'),
    type,
    date: date.toISOString(),
    merchant,
    suggestedCategory: category
  }
}

```

For Command Parsing:

```
function parseCommandFallback(input: string) {
  const lowerInput = input.toLowerCase()

  // Detect type
  const isTransaction = /spent|paid|received|earned/.test(lowerInput)
  const isRenewal = /renewal|bill|subscription|due/.test(lowerInput)

  // Extract amount
  const amountMatch = input.match(/(?:rs\.|₹)\s*(\d+(?:,\d+)*(?:\.\d+)?)/i)
  const amount = parseFloat(amountMatch?.[1]?.replace(/,/g, '') || '0')

  // Extract description
  const words = input.split(/\s+/)
  const description = words.slice(2, 5).join(' ')

  if (isTransaction) {
    return {
      type: 'transaction',
      data: {
        amount,
        type: /received|earned/.test(lowerInput) ? 'INCOME' : 'EXPENSE',
        description,
        date: new Date().toISOString()
      }
    }
  }

  if (isRenewal) {
    return {
      type: 'renewal',
      data: {
        amount,
        name: description,
        frequency: detectFrequency(input)
      }
    }
  }

  return { type: 'unknown' }
}
```

Implementation Steps

1. **Choose AI Provider** (OpenAI recommended)
2. **Install SDK:** `npm install openai`
3. **Add API Key** to `.env.local`:


```
env
OPENAI_API_KEY=sk-...
```
4. **Update** `app/api/ai/parse-sms/route.ts`:
 - Replace `abacusai` import with `openai`
 - Update API call to use OpenAI format
5. **Update** `app/api/ai/parse-command/route.ts`:
 - Same changes as above
6. **Test** SMS parsing and command bar
7. **Verify** dashboard updates after transaction creation

Estimated Time

- OpenAI/Anthropic integration: 2-3 hours
- Rule-based fallback: 4-6 hours
- Testing: 30 minutes

Bug #3: SMS Inbox Syntax Errors



Issue Details

- **Severity:** MEDIUM
- **Status:** ☒ FIXED
- **Impact:** SMS Inbox page failed to load

Errors Found

Error 1: Variable Name with Space

File: app/sms-inbox/page.tsx (Line 50)

Before:

```
const [parsed Transactions, setParsedTransactions] =  
  useState<ParsedTransaction[]>([]);
```

After:

```
const [parsedTransactions, setParsedTransactions] = useState<ParsedTransaction[]>([]);
```

Error 2: Wrong Hook Used

File: app/sms-inbox/page.tsx (Line 56)

Before:

```
useState(() => {  
  fetchCategories();  
});
```

After:

```
useEffect(() => {  
  fetchCategories();  
}, []);
```

Error 3: Missing Import

File: app/sms-inbox/page.tsx (Line 3)

Before:

```
import { useState } from 'react';
```

After:

```
import { useState, useEffect } from 'react';
```

Verification

- ✓ SMS Inbox page now loads successfully
- ✓ UI renders correctly with text area and buttons
- ✓ Message counter works
- ✓ Tips section displays properly

Summary of Fixes

Bug	Severity	Status	Files Changed	Time to Fix
SelectItem Structure	HIGH	✓ Fixed	8 files	30 min
Missing AI Module	CRITICAL	⌚ Pending	2 files	2-3 hours
SMS Inbox Syntax	MEDIUM	✓ Fixed	1 file	5 min

Testing Checklist**After SelectItem Fix**

- [] Clear Next.js cache (`rm -rf .next`)
- [] Restart dev server
- [] Test Add Bill dialog
- [] Test Add Transaction dialog
- [] Test Add Goal dialog
- [] Test Edit dialogs
- [] Verify all dropdowns work

After AI Integration

- [] Test SMS parsing with sample messages
- [] Verify transactions created correctly
- [] Test command bar with natural language
- [] Verify dashboard updates
- [] Test category suggestions
- [] Verify date parsing

Recommendations

1. **Immediate:** Clear cache and rebuild to apply SelectListItem fixes
 2. **High Priority:** Implement OpenAI integration for AI features
 3. **Testing:** Run full regression test suite after fixes
 4. **Documentation:** Update README with AI setup instructions
 5. **Monitoring:** Add error tracking (Sentry) to catch future issues
-

Document Created: January 1, 2026

Last Updated: January 1, 2026

Status: Ready for implementation